

```
#!/usr/bin/env python3
```

```
"""
```

```
CEC UNIFIED CONTROL DASHBOARD v3
```

```
Single Screen Civilization Interface – Governance-Hardened
```

```
v3 closes five structural gaps identified in the v2 audit:
```

1. OBSERVABILITY – `snapshot()/to_dict()` gives every render pass a structured export, so external agents (Claude audit, web views, Gospel-coherence review) consume data instead of parsing stdout.
2. STATUS INTEGRITY – subsystems now report OK / IDLE / NOT_CONNECTED / FAILED instead of collapsing "never wired up" and "broke on use" into the same "[Not connected]" string.
3. CANON ALIGNMENT – world-health scoring is pulled out of the render method into a single named, documented function that is explicitly marked UNRATIFIED pending Governance Bureau lock, rather than silently presented as settled doctrine.
4. INVARIANT SAFETY – `run_live()` now requires an explicit mode ("live" or "demo"). Demo mode operates on a deep-copied registry and can never mutate a live `canon_lock/synthesis_risk` field. Live mode refuses to run at all if `field.step()` isn't real, instead of silently degrading into mock mutation.
5. PROVENANCE – every `render()` call appends an immutable entry to an in-memory audit trail, exportable as JSONL, so the dashboard is append-only like the rest of the stack instead of being the one component with no history.

```
Merges: Registry (population) / Field (relational physics) /  
Events (discontinuities) / Memory (irreversible history / scar tissue)  
"""
```

```
import os
```

```
import sys
```

```
import json
```

```
import copy
```

```
from datetime import datetime, timezone
```

```
from enum import Enum
```

```
from typing import Optional, Any, Dict, List
```

```

# -----
# SUBSYSTEM STATUS — fixes "not connected" vs "failed" ambiguity
# -----
class SubsystemStatus(str, Enum):
    OK = "OK" # connected and returning usable data
    IDLE = "IDLE" # connected, but no data yet (empty, not brok
    NOT_CONNECTED = "NOT_CONNECTED" # never wired up at construction time
    FAILED = "FAILED" # wired up, but errored during use

# --- Graceful imports for existing architecture ---
_IMPORT_ERRORS: Dict[str, str] = {}

try:
    from asset_registry import AssetRegistry
except ImportError as e:
    AssetRegistry = None
    _IMPORT_ERRORS["AssetRegistry"] = str(e)

try:
    from cec_multi_asset_field_v1 import CECAssetField
except ImportError as e:
    CECAssetField = None
    _IMPORT_ERRORS["CECAssetField"] = str(e)

try:
    from cec_event_engine_v2 import CECEventEngine
except ImportError as e:
    CECEventEngine = None
    _IMPORT_ERRORS["CECEventEngine"] = str(e)

try:
    from cec_persistent_world_memory_v1 import WorldMemory, PersistentWorldMemory
except ImportError as e:
    WorldMemory = None
    PersistentWorldMemoryEngine = None
    _IMPORT_ERRORS["PersistentWorldMemoryEngine"] = str(e)

# -----
# CANON SCORING NOTE — world health formula
# -----
# The Feral Gloss Doctrine defines character-level valuation as
#  $V = (F \times G) \times P^n$ 
# World Health is a POPULATION-level aggregate, not a single-asset
# valuation — it is a derived metric, not a direct instantiation of
# the doctrine. Presenting it as canon without Governance Bureau

```

```

# sign-off would be the same citation-integrity failure this system
# already polices at the prose layer (MOD-CITATION-INTEGRITY), just
# committed in code instead of text.
#
# This function makes the doctrine-lineage mapping explicit and
# named so it CAN be audited and ratified – or rejected and replaced
# – without touching render logic:
#
# F (Fidelity) -> mean(1 - synthesis_risk) across population
# G (Governance) -> fraction of population canon_lock == True
# P^n          -> population-scale dampening exponent (n < 1),
#               so health doesn't runaway-scale with headcount
#
# HEALTH_SCORE = (F * G) ** P_EXPONENT, clamped to [0, 1]
#
# STATUS: UNRATIFIED. Swap this function out the moment Keith locks
# the real population-level formula in the Governance Bureau.
CANON_STATUS_HEALTH_FORMULA = "UNRATIFIED – pending Governance Bureau lock"
P_EXPONENT = 0.85

```

```

def compute_world_health(assets: List[Any]) -> float:
    """Population-level health score. See CANON SCORING NOTE above."""
    if not assets:
        return 0.0
    fidelity = sum(1 - getattr(a, "synthesis_risk", 0.5) for a in assets) / len(a)
    governance = sum(1 for a in assets if getattr(a, "canon_lock", False)) / len(a)
    base = max(0.0, fidelity * governance)
    score = base ** P_EXPONENT if base > 0 else 0.0
    return max(0.0, min(1.0, score))

```

```

def _health_status_label(score: float) -> str:
    if score < 0.20:
        return "CRITICAL"
    if score < 0.35:
        return "FRACTURED"
    if score < 0.55:
        return "VOLATILE"
    return "STABLE"

```

```

class CECUnifiedDashboard:
    """
    Unified Control Surface for the CEC synthetic civilization.
    Every layer visible at a glance – and every layer exportable,
    status-honest, canon-traceable, and audit-logged.
    """

```

```
"""
```

```
def __init__(self, registry: Any = None, field: Any = None, memory_engine: Any = None,
              audit_log_path: Optional[str] = None):
    self.registry = registry
    self.field = field
    self.memory_engine = memory_engine
    self.cycle = 0

    # Append-only render audit trail (fix #5)
    self._render_log: List[Dict[str, Any]] = []
    self.audit_log_path = audit_log_path

    # Last-known status per subsystem (fix #2)
    self._status: Dict[str, str] = {
        "registry": SubsystemStatus.NOT_CONNECTED.value,
        "field": SubsystemStatus.NOT_CONNECTED.value,
        "events": SubsystemStatus.NOT_CONNECTED.value,
        "memory": SubsystemStatus.NOT_CONNECTED.value,
    }

    # -----
    # DATA GATHERING (pure – no printing). These are the single source
    # of truth for both the printed render AND snapshot()/to_dict().
    # -----
    def _assets(self) -> List[Any]:
        if not self.registry or not hasattr(self.registry, "_assets"):
            return []
        return list(self.registry._assets.values())

    def _gather_world_health(self) -> Dict[str, Any]:
        assets = self._assets()
        if not self.registry:
            self._status["registry"] = SubsystemStatus.NOT_CONNECTED.value
            return {"status": self._status["registry"], "score": 0.0, "label": "N"}
        if not assets:
            self._status["registry"] = SubsystemStatus.IDLE.value
            return {"status": self._status["registry"], "score": 0.0, "label": "E"}
        try:
            score = compute_world_health(assets)
            self._status["registry"] = SubsystemStatus.OK.value
            return {
                "status": self._status["registry"],
                "score": round(score, 3),
                "label": _health_status_label(score),
                "formula_status": CANON_STATUS_HEALTH_FORMULA,
            }
        except:
```

```

except Exception as e:
    self._status["registry"] = SubsystemStatus.FAILED.value
    return {"status": self._status["registry"], "score": 0.0, "label": "E"}

def _gather_registry(self) -> Dict[str, Any]:
    assets = self._assets()
    if not self.registry:
        return {"status": SubsystemStatus.NOT_CONNECTED.value, "assets": []}
    if not assets:
        return {"status": SubsystemStatus.IDLE.value, "assets": []}
    try:
        rows = []
        for a in assets:
            rows.append({
                "character_id": getattr(a, "character_id", "UNKNOWN"),
                "synthesis_risk": round(getattr(a, "synthesis_risk", 0.0), 3)
                "mavin_price": round(getattr(a, "mavin_price", 0.0), 2),
                "canon_lock": bool(getattr(a, "canon_lock", False)),
            })
        return {"status": SubsystemStatus.OK.value, "assets": rows}
    except Exception as e:
        return {"status": SubsystemStatus.FAILED.value, "assets": [], "error"}

def _gather_field(self) -> Dict[str, Any]:
    if not self.field:
        self._status["field"] = SubsystemStatus.NOT_CONNECTED.value
        return {"status": self._status["field"]}
    assets = self._assets()
    if not assets:
        self._status["field"] = SubsystemStatus.IDLE.value
        return {"status": self._status["field"]}
    try:
        avg_risk = sum(getattr(a, "synthesis_risk", 0.0) for a in assets) / len(assets)
        avg_price = sum(getattr(a, "mavin_price", 0.0) for a in assets) / len(assets)
        out = {
            "status": SubsystemStatus.OK.value,
            "avg_risk": round(avg_risk, 3),
            "avg_price": round(avg_price, 2),
        }
        if hasattr(self.field, "compute_arbitrage_pressure"):
            pressure = self.field.compute_arbitrage_pressure()
            out["high_pressure"] = [k for k, v in pressure.items() if v > 0.0]
        self._status["field"] = SubsystemStatus.OK.value
        return out
    except Exception as e:
        self._status["field"] = SubsystemStatus.FAILED.value
        return {"status": self._status["field"], "error": str(e)}

```

```

def _gather_events(self) -> Dict[str, Any]:
    if not self.field or not hasattr(self.field, "event_engine"):
        self._status["events"] = SubsystemStatus.NOT_CONNECTED.value
        return {"status": self._status["events"], "recent": []}
    try:
        history = getattr(self.field.event_engine, "history", [])
        if not history:
            self._status["events"] = SubsystemStatus.IDLE.value
            return {"status": self._status["events"], "recent": []}
        recent = [{"cycle": e.get("cycle", "?"), "event": e.get("event", "UNK")} for e in history]
        self._status["events"] = SubsystemStatus.OK.value
        return {"status": self._status["events"], "recent": recent}
    except Exception as e:
        self._status["events"] = SubsystemStatus.FAILED.value
        return {"status": self._status["events"], "recent": [], "error": str(e)}

def _gather_memory(self) -> Dict[str, Any]:
    if not self.memory_engine:
        self._status["memory"] = SubsystemStatus.NOT_CONNECTED.value
        return {"status": self._status["memory"]}
    try:
        mem = getattr(self.memory_engine, "memory", self.memory_engine)
        out = {"status": SubsystemStatus.OK.value}
        if hasattr(mem, "current_regime"):
            out["current_regime"] = mem.current_regime
        if hasattr(mem, "fracture_count"):
            out["fracture_count"] = mem.fracture_count
        if hasattr(mem, "get_effective_drift_floor"):
            out["drift_floor"] = round(mem.get_effective_drift_floor(), 3)
        if hasattr(mem, "regime_history") and mem.regime_history:
            last = mem.regime_history[-1]
            out["last_shift"] = {"from": last.get("from"), "to": last.get("to")}
        self._status["memory"] = SubsystemStatus.OK.value
        return out
    except Exception as e:
        self._status["memory"] = SubsystemStatus.FAILED.value
        return {"status": self._status["memory"], "error": str(e)}

# -----
# STRUCTURED EXPORT (fix #1) – single source of truth for consumers
# that are not a human reading a terminal.
# -----

def snapshot(self) -> Dict[str, Any]:
    return {
        "timestamp": datetime.now(timezone.utc).isoformat(timespec="seconds"),
        "cycle": self.cycle,
    }

```

```

        "world_health": self._gather_world_health(),
        "registry": self._gather_registry(),
        "field": self._gather_field(),
        "events": self._gather_events(),
        "memory": self._gather_memory(),
        "subsystem_status": dict(self._status),
    }

def to_dict(self) -> Dict[str, Any]:
    """Alias for snapshot() – for consumers expecting the more common name."""
    return self.snapshot()

def to_json(self, **kwargs) -> str:
    return json.dumps(self.snapshot(), default=str, **kwargs)

# -----
# PRINT LAYER – renders from the same gathered data, never
# re-derives it, so terminal view and export can never drift apart.
# -----
def _print_world_health(self, data: Dict[str, Any]):
    print("\n🌐 WORLD HEALTH")
    print("-----")
    if data["status"] != SubsystemStatus.OK.value:
        print(f"[{data['status']}] {data.get('label', '')}")
        return
    bar_length = 22
    filled = int(data["score"] * bar_length)
    bar = "█" * filled + "░" * (bar_length - filled)
    print(f"[{bar}] {data['score']} → {data['label']}")
    print(f"(formula: {data['formula_status']})")

def _print_registry(self, data: Dict[str, Any]):
    print("\n👥 POPULATION STATE")
    print("-----")
    if data["status"] in (SubsystemStatus.NOT_CONNECTED.value, SubsystemStatus.FAILED.value):
        print(f"[{data['status']}] No assets in registry")
        return
    if data["status"] == SubsystemStatus.FAILED.value:
        print(f"[FAILED] {data.get('error', 'unknown error')}")
        return
    for a in data["assets"]:
        cid = a["character_id"][:16]
        canon = "✅" if a["canon_lock"] else "❌"
        print(f"{cid:<16} | RISK:{a['synthesis_risk']:.2f} | PRICE:${a['mavin']}")

def _print_field(self, data: Dict[str, Any]):
    print("\n🌐 FIELD STATE")

```

```

print("-----")
if data["status"] != SubsystemStatus.OK.value:
    print(f"[{data['status']}]" + (f" {data.get('error')}" if "error" in
    return
print(f"Avg Risk      : {data['avg_risk']:.3f}")
print(f"Avg Price     : ${data['avg_price']:.2f}")
if data.get("high_pressure"):
    print(f"High Pressure: {' , '.join(data['high_pressure'][:3])}")

def _print_events(self, data: Dict[str, Any]):
    print("\n⚡ RECENT EVENTS")
    print("-----")
    if data["status"] != SubsystemStatus.OK.value:
        print(f"[{data['status']}]" + (f" {data.get('error')}" if "error" in
        return
    for e in data["recent"]:
        print(f"Cycle {e['cycle']:>3} → {e['event']}")

def _print_memory(self, data: Dict[str, Any]):
    print("\n📖 WORLD MEMORY (Scar Tissue)")
    print("-----")
    if data["status"] != SubsystemStatus.OK.value:
        print(f"[{data['status']}]" + (f" {data.get('error')}" if "error" in
        return
    if "current_regime" in data:
        print(f"Current Regime : {data['current_regime']}")
    if "fracture_count" in data:
        print(f"Fracture Count : {data['fracture_count']}")
    if "drift_floor" in data:
        print(f"Drift Floor      : +{data['drift_floor']:.3f}")
    if "last_shift" in data:
        ls = data["last_shift"]
        print(f>Last Shift      : {ls['from']} → {ls['to']} (cycle {ls['cycle']

def _print_risk_map(self, registry_data: Dict[str, Any]):
    print("\n🚧 RISK FIELD MAP")
    print("-----")
    if registry_data["status"] in (SubsystemStatus.NOT_CONNECTED.value, Subsy
        SubsystemStatus.FAILED.value):
        print(f"[{registry_data['status']}")
        return
    for a in registry_data["assets"]:
        cid = a["character_id"][:14]
        risk = a["synthesis_risk"]
        bars = int(risk * 18)
        bar = "█" * bars + "░" * (18 - bars)
        print(f"{cid:<14} | {bar} {risk:.2f}")

```



```

# -----
# MAIN RENDER - builds snapshot once, prints from it, logs it.
# -----
def render(self, cycle: Optional[int] = None, clear_screen: bool = True):
    if cycle is not None:
        self.cycle = cycle

    snap = self.snapshot()

    if clear_screen:
        os.system("clear")

    print("=====")
    print("    CEC UNIFIED CONTROL DASHBOARD v3 - SYNTHETIC CIVILIZATION")
    print("=====")
    print(f"TIME: {snap['timestamp']}    |    CYCLE: {snap['cycle']}")
    print("=====")

    self._print_world_health(snap["world_health"])
    self._print_registry(snap["registry"])
    self._print_field(snap["field"])
    self._print_events(snap["events"])
    self._print_memory(snap["memory"])
    self._print_risk_map(snap["registry"])

    print("\n=====")
    print("Commands: [q]uit | [r]efresh | [s]tep field | [e]vent injection")
    print("=====\\n")

    # Fix #5: append-only audit trail
    self._render_log.append(snap)
    if self.audit_log_path:
        try:
            with open(self.audit_log_path, "a") as f:
                f.write(json.dumps(snap, default=str) + "\\n")
        except OSError as e:
            print(f"[AUDIT LOG WRITE FAILED: {e}]")

    return snap

def get_audit_trail(self) -> List[Dict[str, Any]]:
    """Read-only view of every render this instance has produced."""
    return list(self._render_log)

# -----
# LIVE / DEMO RUNNER - fix #4: hard-gated, no implicit mutation of
# live registry state under any circumstance.

```

```

# -----
def _demo_step(self, working_registry: Any, i: int):
    """Mutates ONLY the isolated deep-copy passed in – never self.registry."""
    if not working_registry or not hasattr(working_registry, "_assets"):
        return
    for asset in working_registry._assets.values():
        current_risk = getattr(asset, "synthesis_risk", 0.2)
        asset.synthesis_risk = round(min(0.95, current_risk + 0.015), 3)
        if i % 4 == 0 and hasattr(asset, "canon_lock"):
            if asset.synthesis_risk > 0.45:
                asset.canon_lock = False

def run_live(self, steps: int = 20, mode: str = "live", step_callback=None):
    """
    mode="live": requires a real, connected field with .step(). Will
                  RAISE rather than silently fall back to a mock –
                  a dashboard must never guess whether it's driving
                  real civilization state.
    mode="demo": always operates on a deep-copied registry. The real
                  self.registry is never touched, regardless of what
                  is or isn't connected.
    """
    if mode not in ("live", "demo"):
        raise ValueError("mode must be 'live' or 'demo' – no implicit default")

    if mode == "live":
        if not self.field or not hasattr(self.field, "step"):
            raise RuntimeError(
                "run_live(mode='live') requires a connected field with a real\n"
                "step() method. Refusing to silently substitute mock mutation\n"
                "on what may be live canon-locked civilization state. "\n"
                "Call run_live(mode='demo') if you want a simulated preview."
            )
        print("[LIVE MODE] Driving real field/registry state.\n")
    else:
        print("[DEMO MODE] Operating on an isolated deep-copy. "\n"
              "Live registry/canon_lock state will NOT be touched.\n")

    working_registry = copy.deepcopy(self.registry) if (mode == "demo" and se
    original_registry = self.registry

    for i in range(steps):
        if mode == "demo":
            self._demo_step(working_registry, i)
            self.registry = working_registry # point render at the copy only
        else:
            result = self.field.step()

```

```

        if self.memory_engine and hasattr(self.memory_engine, "process_event"):
            event_name = result.get("event")
            if event_name:
                self.memory_engine.process_event(event_name, self.field,

self.render(cycle=i)

if mode == "demo":
    self.registry = original_registry # restore - real registry was

if step_callback:
    step_callback(i, mode)

try:
    import time
    time.sleep(0.65 if mode == "demo" else 0.75)
except KeyboardInterrupt:
    print("\n[Interrupted]")
    break

# -----
# DEMO / SELF-TEST (runs if executed directly) - explicit demo mode only
# -----
if __name__ == "__main__":
    print("Initializing CEC Unified Dashboard v3 demo...")
    if _IMPORT_ERRORS:
        print(f"[Import notice] Running without: {list(_IMPORT_ERRORS.keys())}")

    registry = None
    field = None
    memory_engine = None

    try:
        if AssetRegistry:
            registry = AssetRegistry()
            if hasattr(registry, "create"):
                registry.create(
                    character_id="CHAR-ISO-BLACK-YOUTH-001",
                    seed_id="CSC-0001",
                    era_variant="Black Era Youth",
                    personality_module="PS-ISO-2.1",
                    behavior_module="BD-ISO-2.3",
                    physical_module="PH-ISO-BLACK-YOUTH-3.1"
                )
            registry.create(
                character_id="CHAR-NOVA-NEON-001",

```

```

        seed_id="CSC-0002",
        era_variant="Neon Drift Youth",
        personality_module="PS-NOVA",
        behavior_module="BD-NOVA",
        physical_module="PH-NOVA-NEON"
    )
except Exception as e:
    print(f"[Warning] Could not fully initialize real registry: {e}")
    registry = None

if not registry or not getattr(registry, "_assets", {}):
    print("[Demo Mode] Using mock assets for dashboard preview.")
    from dataclasses import dataclass, field as dataclass_field
    from typing import List as _List

    @dataclass
    class MockAsset:
        character_id: str
        synthesis_risk: float = 0.18
        mavin_price: float = 95.0
        canon_lock: bool = True
        identity_integrity: float = 0.87
        violation_flags: _List[str] = dataclass_field(default_factory=list)

    class MockRegistry:
        def __init__(self):
            self._assets = {
                "CHAR-ISO-BLACK-YOUTH-001": MockAsset("CHAR-ISO-BLACK-YOUTH-001", 0.18, 95.0, True, 0.87, []),
                "CHAR-NOVA-NEON-001": MockAsset("CHAR-NOVA-NEON-001", 0.31, 6, True, 0.87, []),
                "CHAR-GEN-003": MockAsset("CHAR-GEN-003", 0.24, 81.0, True, 0.87, [])
            }

    registry = MockRegistry()

if CECAssetField and registry:
    try:
        field = CECAssetField(registry)
        if CECEventEngine:
            field.event_engine = CECEventEngine()
    except Exception:
        field = None

if PersistentWorldMemoryEngine:
    try:
        memory_engine = PersistentWorldMemoryEngine()
    except Exception:
        memory_engine = None

```

```
dashboard = CECUnifiedDashboard(registry, field, memory_engine,  
                                audit_log_path="cec_dashboard_audit.jsonl")  
  
# NOTE: self-test always runs demo mode explicitly - a bare `python  
# cec_unified_dashboard_v3.py` must never be able to mutate live state.  
print("\nStarting short demo (press Ctrl+C to stop early)...\n")  
dashboard.run_live(steps=12, mode="demo")  
  
print("\n[Self-test] Snapshot sample:")  
print(dashboard.to_json(indent=2)[:600] + " ...")
```